Copilot no VS Code com cofre Obsidian como memória

Revisão das configurações, unificação dos agentes e protocolo cofre → memória → cofre — teoria, prática e glossário · 20/10/2026

> ***Resumo.*** *Este artigo documenta uma revisão completa da configuração do GitHub Copilot no VS Code 1.142 (Windows). Foram corrigidos erros no* `settings.json` *e no* `mcp.json`*, os agentes customizados espalhados em três pastas foram unificados num único repositório no disco* `D:` *e foi criado um protocolo para que o Copilot consulte o cofre Obsidian antes de agir, use a memória como segunda fonte e salve no cofre o que aprendeu ao terminar cada sessão.*

1. Por que isso importa

Um assistente de código em modo agente decide sozinho quais ferramentas usar, quais arquivos ler e o que alterar. A qualidade desse trabalho depende de três coisas que ficam fora do modelo:

1. **Ferramentas disponíveis** — o que o agente consegue fazer (ler banco, abrir navegador, executar comando). No VS Code, isso vem dos servidores MCP.
2. **Instruções** — como o agente deve se comportar (agentes customizados, arquivos de instruções, prompts).
3. **Contexto persistente** — o que ele já sabe de sessões anteriores. Sem isso, cada conversa começa do zero e o mesmo bug é investigado duas vezes.

A configuração encontrada tinha problemas nas três frentes: servidores MCP que não iniciavam, agentes com cabeçalho inválido (e portanto sem as ferramentas declaradas) e nenhuma memória de longo prazo de fato gravando dados.

2. Glossário

Termo	Significado
MCP (Model Context Protocol)	Protocolo aberto que padroniza como um assistente de IA conversa com ferramentas externas. Um *servidor MCP* expõe *tools* (funções que o modelo pode chamar); o VS Code é o *cliente*.
Servidor stdio	Servidor MCP que roda como processo local; cliente e servidor trocam mensagens JSON-RPC pela entrada e saída padrão. Configurado com `command` e `args`.
Servidor HTTP	Servidor MCP remoto acessado por URL. Não exige instalar nada localmente; pode exigir login (OAuth).
Tool	Uma função exposta por um servidor MCP, como `search_notes` ou `browser_navigate`. Em agentes, referenciada como `servidor/tool` ou `servidor/*` (todas as tools do servidor).
Handshake `initialize`	Primeira mensagem do protocolo MCP. Se o servidor não responde a ela, o cliente considera que ele não iniciou.
Agente customizado (`*.agent.md`)	Arquivo Markdown que define uma persona do Copilot: nome, descrição, modelo, lista de tools permitidas e instruções. Substituiu os antigos *chat modes* (`*.chatmode.md`).
Prompt file (`*.prompt.md`)	Prompt reutilizável invocado com `/nome` no chat.
Instructions file (`*.instructions.md`)	Regras aplicadas automaticamente às conversas. O campo `applyTo` (glob) define a quais arquivos se aplicam; `'**'` aplica a tudo.
Frontmatter	Bloco YAML entre linhas `---` no início do arquivo Markdown. Precisa começar na **primeira linha**; qualquer coisa antes dele (linha em branco, cerca de código) faz o VS Code ignorá-lo.
Hook	Comando que o VS Code executa automaticamente em eventos do ciclo de vida do agente (`SessionStart`, `Stop`, `PreToolUse` …). Recebe JSON pela entrada padrão e responde JSON pela saída.
Harness	O "motor" que executa o agente. O VS Code tem o harness local (Local) e o do Copilot; cada um lê customizações de pastas um pouco diferentes.
JSONC	JSON com comentários e vírgulas finais tolerados. É o formato do `settings.json`. Tolerado não significa correto: chaves duplicadas são ignoradas silenciosamente.
Junction	Atalho de pasta do NTFS (Windows). Programas que abrem `C:\...\agents` enxergam, na verdade, o conteúdo de `D:\...\agents`. Equivale a um link simbólico de diretório, sem exigir privilégio de administrador.
Cofre (vault)	Pasta de notas Markdown do Obsidian. Aqui: `C:/Users/Raf_F/Documents/obsidian/rfsprod`.
Grafo de memória	Banco de entidades, relações e observações mantido pelo servidor MCP `memory`, gravado num arquivo JSONL.
npx / uvx	Executores que baixam e rodam um pacote sob demanda: `npx` para pacotes npm (Node.js), `uvx` para pacotes Python.

3. Visão geral da arquitetura final

```
FONTE ÚNICA:  D:\devEnv\git_ai\upgrading-ai-agentic\
              ├─ agents\        6 agentes (*.agent.md)
              ├─ prompts\       2 prompts (*.prompt.md)
              ├─ instructions\  protocolo do cofre (*.instructions.md)
              ├─ hooks\         SessionStart / Stop
              └─ memory\        grafo do MCP memory (memory.jsonl)

QUEM APONTA PARA ELA:
  settings.json  chat.agentFilesLocations ........ ► agents\
                 chat.promptFilesLocations ....... ► prompts\
                 chat.instructionsFilesLocations . ► instructions\
  junction       ~\.copilot\agents ............... ► agents\
  junction       ~\.copilot\instructions ......... ► instructions\
  junction       ~\.copilot\hooks ................ ► hooks\
  mcp.json       memory (MEMORY_FILE_PATH) ....... ► memory\

MCP (mcp.json):
  obsidian-vault  lê o cofre       cofre-fs  grava no cofre
  memory          grafo            fetch, playwright, context7,
                                   microsoft-learn, mysql-local, scriptos...
```

Princípio: uma única fonte de verdade versionada (`D:`), com as pastas padrão do Copilot apontando para ela, em vez de cópias divergentes em três lugares.

4. Parte 1 — `settings.json`

Arquivo: `%APPDATA%\Code\User\settings.json`

Cada chave foi conferida contra o `package.json` da extensão Copilot embutida no VS Code 1.142 (a partir desta versão o Copilot vem dentro do VS Code, em `resources\app\extensions\copilot` , e não aparece mais em `code --list-extensions`).

Problema	Por que é um problema	Correção
`github.copilot.chat.agentDebugLog.fileLogging.enabled` declarada duas vezes	Em JSONC a última ocorrência vence sem aviso; mascara edições futuras	Mantida uma só
Vírgula após o último perfil de terminal	Tolerada em JSONC, mas quebra qualquer ferramenta que leia o arquivo como JSON estrito	Removida
`github.copilot.chat.agent.enabled`	Chave não existe mais; o modo agente é controlado por `chat.agent.enabled`	Removida

Adicionadas na etapa de unificação:

```jsonc
"chat.instructionsFilesLocations": {
  "D:\\devEnv\\git_ai\\upgrading-ai-agentic\\instructions": true
},
"chat.useHooks": true,          // executa hooks do Copilot
"chat.useClaudeHooks": false    // NÃO executa os hooks do Claude Code (~/.claude/settings.json)
```

> ***Atenção:*** *`chat.agentFilesLocations`, `chat.promptFilesLocations` e `chat.instructionsFilesLocations` são marcadas como deprecated na documentação, mas continuam funcionando no harness local da 1.142. Por isso foram mantidas **e** reforçadas com junctions nas pastas padrão (seção 7) — se as configurações forem removidas numa versão futura, as junctions continuam valendo.*

5. Parte 2 — `mcp.json`

Arquivo: `%APPDATA%\Code\User\mcp.json` — lista os servidores MCP disponíveis em qualquer workspace.

5.1 Teoria: como um servidor stdio é iniciado

O VS Code executa `command` com `args`, escreve a mensagem `initialize` na entrada padrão do processo e espera a resposta. Se o processo fecha antes de responder ("Connection closed"), o pacote não existe, ou o script não chama o laço do servidor, as tools nunca aparecem no chat — e o erro só é visível no log de saída do MCP.

5.2 Diagnóstico e correções

Servidor	Problema encontrado	Correção	Teste
`fetch`	Pacote `@modelcontextprotocol/server-fetch` **não existe** no npm (o servidor oficial é Python)	`uvx mcp-server-fetch`	✅ tool `fetch`
`scriptos`	`exec_commands.py` define as tools mas nunca chama `mcp.run()`; o processo termina imediatamente	Executar via CLI do FastMCP: `fastmcp.exe run exec_commands.py`	✅ 3 tools
`playwright`	`--no-install` dependia da versão global antiga (0.0.79)	`npx -y @playwright/mcp@latest`	—
`context7`	Lista `tools` citava `get-library-docs`, renomeada para `query-docs`	Endpoint HTTP `https://mcp.context7.com/mcp` (mesmo do plugin do Claude Code)	—
`memory-cht-reader`	Caminho com outro usuário (`C:\Users\usuario\...`)	`%APPDATA%\Code\User\workspaceStorage` (do usuário correto)	—
`docs-reader`	Caminho `"."` depende do diretório em que o processo nasce	`${workspaceFolder}`	—
Todos	Campo `tools` não faz parte do schema do `mcp.json` e tinha nomes errados (`browser_screenshot` em vez de `browser_take_screenshot`)	Removido. A seleção de tools é feita no seletor do chat ou no frontmatter do agente	—
`memory`	Sem caminho definido, o grafo ia para dentro do cache do npx (perdível); nenhum dado havia sido salvo	`env.MEMORY_FILE_PATH = D:\...\memory\memory.jsonl`	✅ 9 tools
`mysql-local`	Correto	Mantido (modo somente leitura por padrão)	✅ 2 tools

5.3 Servidores reaproveitados e novos

Servidor	Origem	Função
`atlassian` (HTTP)	Configuração do Claude Code	Jira e Confluence; pede login OAuth no primeiro uso
`obsidian-vault` (stdio)	Configuração do Claude Code	**Lê** o cofre: `search_notes`, `read_notes`
`cofre-fs` (stdio)	Novo	`@modelcontextprotocol/server-filesystem` restrito à pasta do cofre; **grava** notas (`write_file`, `edit_file`)

O `obsidian-vault` só tem tools de leitura; sem um servidor de escrita, a etapa "salvar no cofre" seria impossível. O `cofre-fs` só enxerga a pasta do cofre — não consegue escrever em nenhum outro lugar do disco.

5.4 Prática: testar um servidor sem abrir o VS Code

O MCP Inspector tem um modo de linha de comando que faz o handshake e lista as tools:

```
npx -y @modelcontextprotocol/inspector --cli `
    npx -y @modelcontextprotocol/server-filesystem C:\Users\Raf_F\Documents\obsidian\rfsprod `
    --method tools/list
```

Se a saída trouxer a lista de tools, o servidor está saudável. "Connection closed" indica que o processo morreu antes do `initialize`.

6. Parte 3 — Unificação dos agentes

6.1 Situação encontrada

Os agentes estavam em três lugares, com versões diferentes do mesmo agente:

Local	Conteúdo
`%APPDATA%\Code\User\prompts` (perfil do VS Code)	5 agentes
`%APPDATA%\Code\User` (raiz, fora de qualquer pasta lida)	1 agente solto, ignorado
`C:\Users\Raf_F\.copilot\agents`	1 agente (`modernize-java`)
`D:\devEnv\git_ai\upgrading-ai-agentic\agents`	1 agente (`ui-generator`, versão reduzida)

6.2 Teoria: por que frontmatter inválido é grave

O VS Code só reconhece o frontmatter se o arquivo **começar** com `---`. Quando o cabeçalho é ignorado, o agente perde nome, descrição e, principalmente, a lista `tools` — passa a rodar sem as ferramentas para as quais foi escrito, ou com tools padrão diferentes das esperadas. Os erros encontrados:

Erro	Exemplo	Efeito
Cerca de código em volta do arquivo	```` ```yaml ```` na linha 1	Frontmatter não reconhecido
Linha em branco antes do `---`	`\n---`	Frontmatter não reconhecido
Escapes de Markdown	`\---`, `\#`, `\*`	Cabeçalho e títulos viram texto literal
Lista YAML com `*`	`* memory/*`	Em YAML, `*` inicia um *alias*; a lista é inválida

6.3 Resultado

Um script Python normalizou os arquivos (removeu cercas e escapes, reconstruiu o frontmatter com lista YAML válida) e gravou tudo em `D:\...\agents`:

Agente final	Origem	Observação
`bugs-killer`	perfil	Frontmatter reconstruído
`smoke-test-web`	`smoke-test` (r5)	Renomeado: smoke test genérico de site
`smoke-test-angular`	`smoke-test-r9` (r10)	Renomeado: smoke test Angular com controle de processos via `scriptos`
`ui-generator`	`ui-generator` + `ui-generator2`	Mantida a versão completa (a antiga era subconjunto)
`ux-navigator`	perfil	Escapes removidos
`modernize-java`	`~/.copilot/agents`	Preservado integralmente, inclusive `handoffs`

Todos receberam no `tools`: `obsidian-vault/*`, `cofre-fs/*`, `memory/*` e `memory-cht-reader/*`.

> ***Regra prática:*** *um agente com `tools:` declarado só usa as tools listadas. Instruções globais pedindo "consulte o cofre" não adiantam se o agente não tem a tool do cofre na lista.*

7. Parte 4 — Uma fonte só: o disco `D:`

7.1 Teoria

O Copilot procura customizações de usuário em pastas padrão (`~/.copilot/agents`, `~/.copilot/instructions`, `~/.copilot/hooks`) e na pasta de perfil do VS Code. Copiar arquivos para cada uma gera divergência. A solução é manter os arquivos em um lugar (o repositório em `D:`, versionado no Git) e fazer as pastas padrão **apontarem** para ele.

7.2 Prática

```
$d = 'D:\devEnv\git_ai\upgrading-ai-agentic'
foreach ($n in 'agents','instructions','hooks') {
    New-Item -ItemType Junction -Path "$HOME\.copilot\$n" -Target "$d\$n"
}
```

Pasta padrão	Aponta para
`C:\Users\Raf_F\.copilot\agents`	`D:\devEnv\git_ai\upgrading-ai-agentic\agents`
`C:\Users\Raf_F\.copilot\instructions`	`D:\devEnv\git_ai\upgrading-ai-agentic\instructions`
`C:\Users\Raf_F\.copilot\hooks`	`D:\devEnv\git_ai\upgrading-ai-agentic\hooks`

A pasta de perfil `%APPDATA%\Code\User\prompts` foi esvaziada (com backup em `Code\User\backup-agentes-20261020-173330`).

8. Parte 5 — Protocolo cofre → memória → cofre

8.1 Teoria: memória em camadas

Camada	Onde	Natureza	Uso
1. Cofre Obsidian	`rfsprod`	Notas legíveis, ligadas por `[[wikilinks]]`, com frontmatter	Fonte primária: decisões, soluções, conhecimento do projeto
2. Grafo de memória	`memory.jsonl`	Entidades e observações curtas	Fatos pontuais quando o cofre não tem
3. Histórico de chats	`workspaceStorage`	Conversas brutas do VS Code	Último recurso

O cofre é a camada principal porque é durável, pesquisável por humanos e por agentes, e já segue um padrão (`_CLAUDE.md`) pensado para leitura por IA: cada nota começa com a seção `## For future agent`, tem metadados filtráveis e marca o nível de confiança das afirmações.

8.2 A instrução global

Arquivo: `instructions/cofre-memoria.instructions.md` (com `applyTo: '**'`, vale para toda conversa)

1. **Antes de agir:** buscar no cofre com `search_notes` (pelo menos dois termos) e abrir as notas com `read_notes`.
2. **Se não houver:** `memory/search_nodes`, depois `memory-cht-reader`.
3. **Ao terminar:** gravar com `cofre-fs` o que for reutilizável:
   - `Dev Logs/AAAA-MM-DD - assunto.md` para trabalho técnico;
   - `Knowledge/` para conhecimento permanente;
   - `Projects/` para estado de projeto;
   - uma linha no log diário `Logs/AAAA-MM-DD.md`.

Não se salva conversa trivial nem segredos. Alterar nota existente com base em fonte externa, apagar ou arquivar sempre exigem confirmação.

8.3 Os hooks

Arquivos: `hooks/cofre.json` e `hooks/scripts/cofre-hook.ps1`

Instruções são sugestões ao modelo; hooks são executados pelo VS Code, independentemente do que o modelo decidir. Os dois se complementam:

```
Primeira mensagem
  ↳ SessionStart
       ├─ grava o horário de início em %TEMP%\copilot-cofre\<sessão>.start
       └─ injeta o lembrete do protocolo (additionalContext)

Fim de cada resposta
  ↳ Stop
       ├─ stop_hook_active = true? ..................... libera
       ├─ já cobrou nesta sessão (<sessão>.asked)? ...... libera
       ├─ alguma nota do cofre alterada desde o início?  libera
       └─ senão ► {"decision": "block", "reason": "salve no cofre
                   ou responda 'Cofre: nada novo a salvar'"}
                   (uma única vez por sessão)
```

Detalhes de projeto:

- `stop_hook_active` vem do VS Code quando o agente já está continuando por causa de um bloqueio anterior; checá-lo evita loop infinito.
- **Uma cobrança por sessão:** o evento `Stop` dispara ao fim de **cada resposta**, não ao fim da conversa. Sem o arquivo-marcador `.asked`, toda resposta geraria uma rodada extra (e consumo de créditos).
- **Detecção de gravação:** compara o `LastWriteTime` das notas `.md` do cofre (ignorando `.obsidian`, `.trash` e `.claude`) com o horário de início da sessão.
- **Limpeza:** marcadores com mais de 7 dias são apagados no `SessionStart`.

8.4 Prática: testar os hooks sem o VS Code

```powershell
$h = 'D:\devEnv\git_ai\upgrading-ai-agentic\hooks\scripts\cofre-hook.ps1'
function Hook($ev, $json) {
    $json | powershell -NoProfile -ExecutionPolicy Bypass -File $h -Event $ev
}
Hook SessionStart '{"session_id":"teste-1"}'
Hook Stop '{"session_id":"teste-1","stop_hook_active":false}'  # bloqueia
Hook Stop '{"session_id":"teste-1","stop_hook_active":true}'   # libera
Hook Stop '{"session_id":"teste-1"}'                           # libera (já cobrou)
```

Resultado obtido: lembrete injetado, um bloqueio, depois duas liberações — o comportamento esperado.

9. Operação do dia a dia

Quero...	Faça
Criar um agente novo	Crie `D:\...\agents\<nome>.agent.md`; frontmatter na linha 1; inclua as quatro tools do cofre e da memória em `tools`
Criar um prompt reutilizável	`D:\...\prompts\<nome>.prompt.md`
Ver se o Copilot carregou tudo	Paleta de comandos → **Chat: Configure Instructions** / **Configure Custom Agents**
Reiniciar os servidores MCP	Paleta → **MCP: List Servers** → *Restart*
Desligar a cobrança de fim de sessão	Remova o bloco `Stop` de `hooks/cofre.json`
Desligar todos os hooks	`"chat.useHooks": false` no `settings.json`
Restaurar a configuração anterior	Backups `settings.json.bak-20261020-170727`, `mcp.json.bak-20261020-170727` e pasta `backup-agentes-20261020-173330`, todos em `%APPDATA%\Code\User`

10. Riscos e pendências

1. **`.env` versionado em repositório público.** O arquivo `.env` de `upgrading-ai-agentic` é rastreado pelo Git e o repositório é público no GitHub. Se contiver credenciais, elas devem ser trocadas e o arquivo retirado do controle de versão (`git rm --cached .env` e inclusão no `.gitignore`).
2. **`mysql-local` conecta como `root` sem senha.** Está em modo somente leitura, mas o recomendado é um usuário dedicado de leitura.
3. **Possível duplicidade de instruções.** A instrução é lida pela junction e pela configuração `chat.instructionsFilesLocations`. Se aparecer duas vezes, remova a configuração.
4. **Configurações *deprecated*.** As `chat.*FilesLocations` podem sumir em versões futuras; as junctions cobrem esse cenário.
5. **`scriptos`.** Adicionando `if __name__ == "__main__": mcp.run()` ao fim de `exec_commands.py`, ele volta a poder ser executado direto pelo `python.exe`.
6. **Mudanças não commitadas.** Tudo em `D:` está pendente no Git para revisão.

11. Referências

- VS Code — Agent hooks: https://code.visualstudio.com/docs/copilot/customization/hooks
- VS Code — Local hooks reference: https://code.visualstudio.com/docs/agents/reference/hooks-reference
- VS Code — Custom agents: https://code.visualstudio.com/docs/copilot/customization/custom-agents

- VS Code — Custom instructions: https://code.visualstudio.com/docs/copilot/customization/custom-instructions
- VS Code — Prompt files: https://code.visualstudio.com/docs/copilot/customization/prompt-files
- Model Context Protocol: https://modelcontextprotocol.io
- MCP Inspector: https://github.com/modelcontextprotocol/inspector
- Context7 MCP: https://github.com/upstash/context7
- Playwright MCP: https://github.com/microsoft/playwright-mcp